

GPT Clone Backend Documentation (MVP Version)

PROJECT OVERVIEW

Ye project ChatGPT jaisa AI Chat Application hai.

Current phase me backend MVP complete hai.

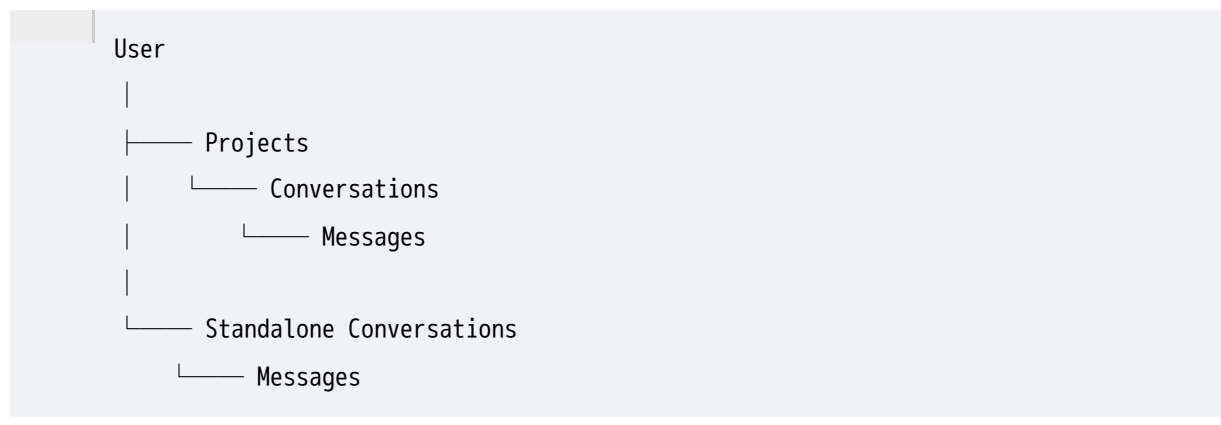
AI Provider:

- OpenRouter

Supported Models:

- Claude Sonnet 4
- Gemini
- DeepSeek
- Future OpenRouter Models

Architecture:



Tech Stack

Backend

- Node.js
- Express.js
- TypeScript

Database

- PostgreSQL (Supabase)

ORM

- Prisma ORM

Authentication

- JWT Access Token

- JWT Refresh Token
- HttpOnly Cookies

Validation

- Zod

AI

- OpenRouter SDK

Streaming

- Server Sent Events (SSE)

Authentication System

Implemented Features

REGISTER

POST

```
/api/auth/register
```

Creates new user.

LOGIN

POST

```
/api/auth/login
```

Response:

```
{
  "id": "...",
  "name": "...",
  "email": "...",
  "role": "USER"
}
```

Cookies:

- accessToken
- refreshToken

LOGOUT

POST

```
/api/auth/logout
```

Clears cookies.

Deletes refresh token.

REFRESH TOKEN

POST

```
/api/auth/refresh-token
```

Generates new access token.

CURRENT USER

GET

```
/api/auth/me
```

Returns authenticated user.

User Roles

Enum:

```
enum UserRole {  
    USER  
    ADMIN  
}
```

Current Usage:

- USER
 - ADMIN
-

AI Model System

Purpose:

Admin dynamically manage available AI models.

Users never see actual OpenRouter model value.

Frontend sends modelId only.

DATABASE MODEL

```
model AIModel
```

Fields:

- id
- label
- value
- provider
- description
- isDefault
- isActive

Example:

```
{  
  "label": "Claude Sonnet 4",  
  "value": "anthropic/claude-sonnet-4"  
}
```

AI Model APIs

CREATE

POST

```
/api/models
```

GET ALL

GET

```
/api/models
```

GET BY ID

GET

```
/api/models/:id
```

UPDATE

PATCH

```
/api/models/:id
```

DELETE

DELETE

```
/api/models/:id
```

Project System

Projects act as folders.

Example:

```
React Project
Node Project
Company Project
```

Each project contains multiple conversations.

DATABASE

```
model Project
```

Fields:

- id
 - name
 - userId
-

Project APIs

CREATE PROJECT

POST

```
/api/projects
```

GET PROJECTS

GET

```
/api/projects
```

GET PROJECT

GET

```
/api/projects/:id
```

UPDATE PROJECT

PATCH

```
/api/projects/:id
```

DELETE PROJECT

DELETE

```
/api/projects/:id
```

Cascade Delete:

```
Project Deleted
↓
All Conversations Deleted
↓
All Messages Deleted
```

Conversation System

Conversation = Single Chat

Example:

```
How to learn React
Node Architecture Discussion
Resume Review
```

Important Design Decision

Model is locked per conversation.

User selects model once.

Conversation always uses same model.

No model switching inside conversation.

Reason:

- Better context consistency

- Better AI responses
- Simpler architecture

Conversation Types

STANDALONE CONVERSATION

No project.

	Sidebar Chat
--	--------------

PROJECT CONVERSATION

Inside project.

	Project
	└ Conversation

TEMPORARY CONVERSATION

Not shown in sidebar.

Hidden from conversation list.

Used like ChatGPT Temporary Chat.

Field:

	isTemporary Boolean
--	---------------------

Conversation Database

	model Conversation
--	--------------------

Fields:

- id
- title
- userId
- projectId
- modelId
- isTemporary
- createdAt
- updatedAt

Conversation APIs

CREATE CONVERSATION

POST

```
/api/conversations
```

Payload:

```
{
  "modelId": "...",
  "projectId": "...",
  "isTemporary": false
}
```

GET CONVERSATIONS

GET

```
/api/conversations
```

Returns standalone conversations only.

Temporary chats hidden.

GET PROJECT CONVERSATIONS

GET

```
/api/projects/:projectId/conversations
```

GET CONVERSATION

GET

```
/api/conversations/:id
```

DELETE CONVERSATION

DELETE

```
/api/conversations/:id
```

Cascade:


```
Conversation Deleted
```



```
Messages Deleted
```

Message System

Every chat contains messages.

Message Roles

```
enum MessageRole {  
    USER  
    ASSISTANT  
    SYSTEM  
}
```

Current Usage:

- USER
 - ASSISTANT
-

Message Database

```
model Message
```

Fields:

- id
- conversationId
- role
- content

Token Tracking:

- promptTokens
 - completionTokens
 - totalTokens
-

Message API

GET MESSAGES

GET

```
/api/conversations/:id/messages
```

Returns full chat history.

OpenRouter Integration

Provider:

```
OpenRouter
```

Model comes from:

```
Conversation
  ↓
  modelId
  ↓
  AIModel
  ↓
  value
```

Example:

```
anthropic/claude-sonnet-4
```

Chat Architecture

CONTEXT STRATEGY

Every request sends:

```
Previous Messages
+
Current User Message
```

to OpenRouter.

Current Limit:

```
OPENROUTER_VALUES.PREVIOUS_MESSAGE
```

Normal Chat API

POST

```
/api/chat/send
```

Flow:

```
graph TD; A[Validate Conversation] --> B[Save USER Message]; B --> C[Load Previous Messages]; C --> D[Call OpenRouter]; D --> E[Save ASSISTANT Message]; E --> F[Return Response];
```

Streaming Chat API

POST

```
/api/chat/stream
```

Uses:

```
SSE  
(Server Sent Events)
```

Flow

```
graph TD; A[Validate Conversation] --> B[Save USER Message]; B --> C[Load Previous Messages]; C --> D[OpenRouter Stream]; D --> E[Stream Chunks]; E --> F[Save ASSISTANT Message];
```

↓
Return done: **true**

Streaming Response Format

Chunk:

```
{  
  "content": "Hello"  
}
```

Completion:

```
{  
  "done": true  
}
```

Token Tracking

Streaming stores:

```
promptTokens  
completionTokens  
totalTokens
```

Data comes from:

```
chunk.usage
```

Example:

```
{  
  "prompt_tokens": 975,  
  "completion_tokens": 392,  
  "total_tokens": 1367  
}
```

Automatic AI Title Generation

Every new conversation starts with:

```
New Chat
```

After first user message:

```
Generate title
```

↓

```
Save title
```

Example:

Input:

```
How to learn React Hooks
```

Generated Title:

```
React Hooks Learning Guide
```

Constants

Important Constants:

```
OPENROUTER_VALUES.MAX_TOKEN
```

Maximum response tokens.

```
OPENROUTER_VALUES.PREVIOUS_MESSAGE
```

History message count.

```
OPENROUTER_VALUES.CONVERSATION_DEFAULT_TITLE
```

Default conversation title.

Security

Implemented

HTTPONLY COOKIES

Access Token

Refresh Token

CORS

Allowed Frontend:

```
localhost:5173
localhost:5174
localhost:5175
```

JWT MIDDLEWARE

Authenticated routes protected.

OWNERSHIP VALIDATION

Every resource checks:

```
userId
```

before access.

Users cannot access others' data.

Current MVP Status

Completed

- ✓ Authentication
 - ✓ AI Models CRUD
 - ✓ Projects CRUD
 - ✓ Conversations CRUD
 - ✓ Messages Listing
 - ✓ OpenRouter Integration
 - ✓ Non Streaming Chat
 - ✓ Streaming Chat
 - ✓ Token Tracking
 - ✓ AI Title Generation
 - ✓ Temporary Chat
-

Planned Future Features

Phase 2

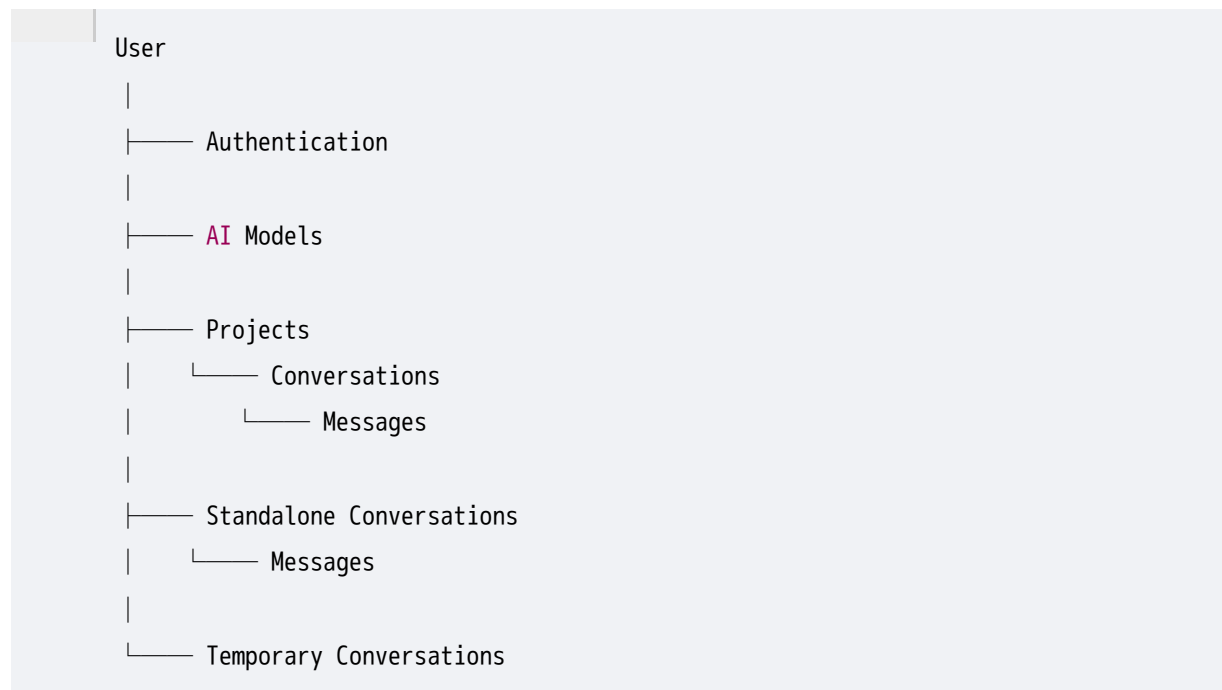
- Conversation Rename
- Usage Analytics

- Project Instructions
- Client Disconnect Handling
- Better Streaming Events

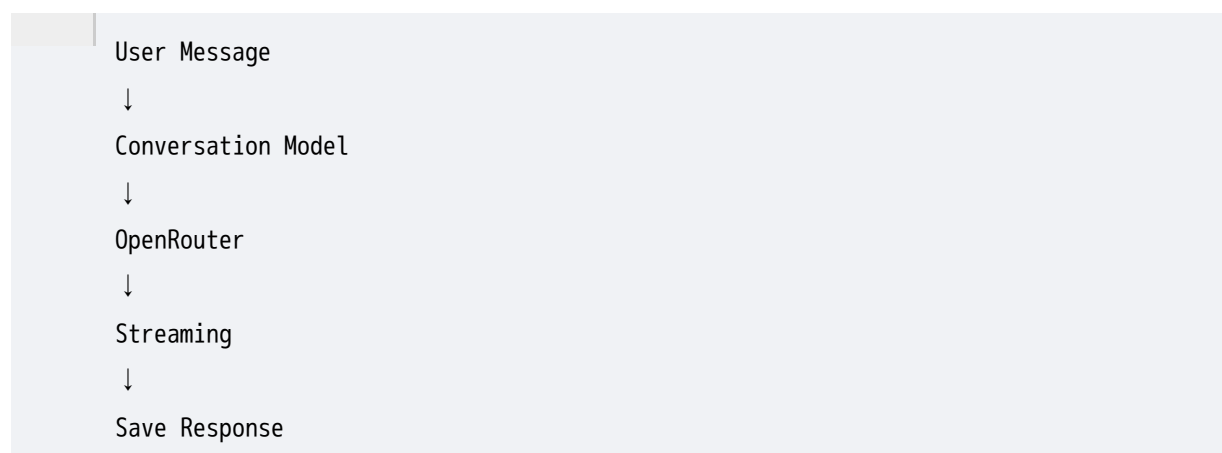
Phase 3

- File Upload
- Image Upload
- Vision Models
- Image Generation
- Web Search
- Voice Chat

Final Architecture Summary



AI Flow



↓
Save Tokens
↓
Generate Title

Current Backend State:

MVP COMPLETE

READY FOR FRONTEND INTEGRATION

READY FOR PRODUCTION-LIKE TESTING

